



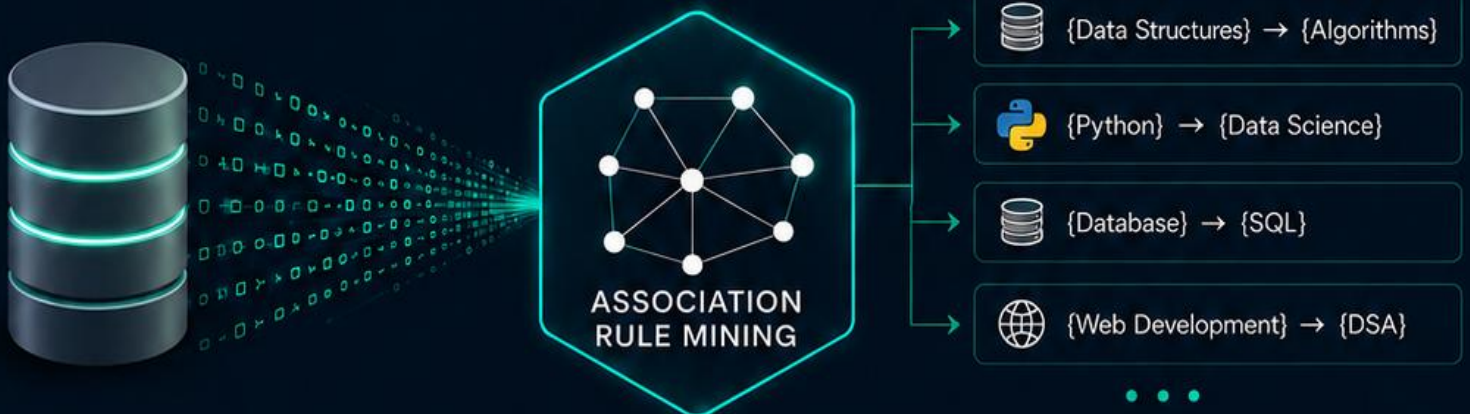
Dezyne E'cole College
Business • Design • Computers

PROJECT REPORT

Association Rule Mining Project

based on

Student Enrollment Patterns in
Programming Courses



NAME

Digvijay Singh Parmar



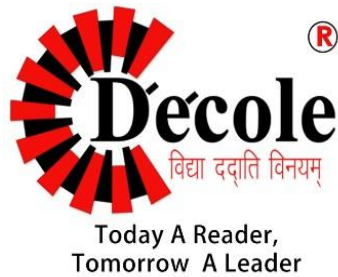
DEPARTMENT

BCA VI Sem



COLLEGE

Dezyne École College



Dezyne École College

Title of Assignment

Association Rule in Data Mining

Based on Student Enrolment Patterns

Subject Name

Data Mining with R

Submitted By

Digvijay Singh Parmar

Department- BCA

Semester- 6th

Session- 2025-2026

Project Report on
Association Rule in Data Mining

Based on Student Enrolment Patterns

Submitted To

Dezyne École College

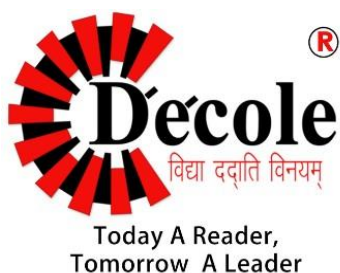
Towards the Partial Fulfilment of

6th

Semester

By

Digvijay Singh Parmar



www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,

Digvijay Singh Parmar

Department of BCA 6th Semester

Dezyne École College, Ajmer

Contents

Acknowledgement	
Section 1: Introduction — What is Market Basket Analysis?	1
1.1 Key Definitions	5
2.1 The Transaction Database	7
2.2 Step 1 — Count Individual Item Frequencies	4
2.3 Step 2 — Generate and Count 2-Item Candidates	4
Section 3: The Three Key Metrics — Support, Confidence, Lift	6
3.1 Support	6
3.2 Confidence	7
3.3 Lift — The Most Important Metric	7
Section 4: The Apriori Algorithm — Full Step-by-Step Walkthrough	9
4.1 The Core Principle	9
4.2 Complete Algorithm with Our Example	10
4.3 Rule Generation from Frequent Itemsets	10
4.4 Time & Space Complexity	11
Section 5: All Mined Rules — Confidence, Lift & Amazon Precision	12
5.1 Complete Rules Table for Grocery Example	12
Section 6: Practical Example 2 — Amazon Electronics (5 Transactions)	14
6.1 Transaction Database	14
6.2 1-Itemset Frequencies	15
6.3 Top Rules with High Lift	15
Section 7: Practical Example 3 — Manual Apriori (Exam-Style)	17
7.1 The Given Database	17
7.2 Pass 1 — Frequent 1-Itemsets	17
7.3 Pass 2 — Frequent 2-Itemsets	18
7.4 Pass 3 — Frequent 3-Itemsets	18
7.5 Rule Generation (MinConf = 75%)	19
Section 8: FP-Growth vs Apriori — Comparison	20
8.1 FP-Tree Construction	20
8.2 FP-Tree for Our Grocery Example	20
8.3 Comparison Table	21
Section 9: Industry Application — How Amazon Uses Association Rules	22
9.1 Where Association Rules Appear on Amazon	22
9.2 The Full Amazon Pipeline	23

Section 1: Introduction — What is Market Basket Analysis?

Market Basket Analysis — How It Works



Stronger rules = better insights for merchandising, promotions & recommendations. | Data Mining | Market Basket Analysis

The Business Problem

Amazon processes hundreds of millions of transactions every day. Hidden inside these purchases are patterns: customers who buy a laptop almost always buy a mouse. Customers who buy bread frequently buy butter. If Amazon can discover these patterns automatically, it can display a 'Frequently Bought Together' recommendation panel — increasing average order value without any human curation.

Association Rule Mining is the algorithmic technique that discovers these patterns. Given a database of transactions, it finds rules of the form:

M → N (If a customer buys M, they are likely to also buy N)

and assigns each rule three scores that measure how useful it is.

1.1 Key Definitions

Term	Formal Definition	Intuitive Meaning	Example
Transaction	A single shopping session with a set of items purchased	One customer's cart	T1 = {Rice, Juice, Jam}
Itemset	Any subset of items	A group of products	{ Rice, Jam }
Frequent Itemset	An itemset whose support $\geq$ minimum threshold	Appears in enough baskets	{ Rice, Jam } in 4/10 baskets
Association Rule	An implication $M \rightarrow N$ derived from transactions	If-then pattern	Rice $\rightarrow$ Jam
Support	Fraction of transactions containing the itemset	How common is this combination?	support = $4/10 = 40\%$
Confidence	Fraction of M-transactions that also contain N	How reliable is the rule?	conf = $4/6 = 67\%$
Lift	Confidence divided by support of N	Is the rule better than random?	lift = $0.67/0.60 = 1.11$

Section 2: Practical Example 1 — Grocery Store (10 Transactions)

2.1 The Transaction Database

Setup: A grocery store records 10 customer baskets. Each basket is one transaction. We set minimum support = 40% (an itemset must appear in at least 4 out of 10 transactions to be considered 'frequent') and minimum confidence = 60%.

Transaction	Items Purchased	No. of Items
T01	{Rice, Juice, Jam}	3
T02	{Rice, Jam, Cookies}	3
T03	{Juice, Cookies, Chips}	3
T04	{Rice, Juice, Jam, Cookies}	4
T05	{Rice, Juice}	2
T06	{Jam, Cookies, Chips}	3
T07	{Rice, Jam, Cookies, Juice}	4
T08	{Juice, Chips, Chocolate}	3
T09	{Rice, Cookies, Chocolate}	3
T10	{Jam, Juice, Chips, Chocolate}	3

2.2 Step 1 — Count Individual Item Frequencies

Scan all 10 transactions and count how many times each item appears. Compare with the minimum support threshold (4 occurrences = 40%).

Item	Appears in transactions	Count	Support	Frequent?
Rice	T01,T02,T04,T05,T07,T09	6	6/10 = 60%	YES ✓
Juice	T01,T03,T04,T05,T07,T08,T10	7	7/10 = 70%	YES ✓
Jam	T01,T02,T04,T06,T07,T10	6	6/10 = 60%	YES ✓
Cookies	T02,T03,T04,T06,T07,T09	6	6/10 = 60%	YES ✓
Chips	T03,T06,T08,T10	4	4/10 = 40%	YES ✓
Chocolate	T08,T09,T10	3	3/10 = 30%	NO — pruned X

Apriori Principle applied: Chocolate has support 30% < 40% threshold. This means we NQVQR need to check any 2-itemset containing Chocolate — {Rice, Chocolate}, {Juice, Chocolate}, etc. are all automatically infrequent. We prune them without even computing their support. This is the power of the Apriori Principle.

2.3 Step 2 — Generate and Count 2-Item Candidates

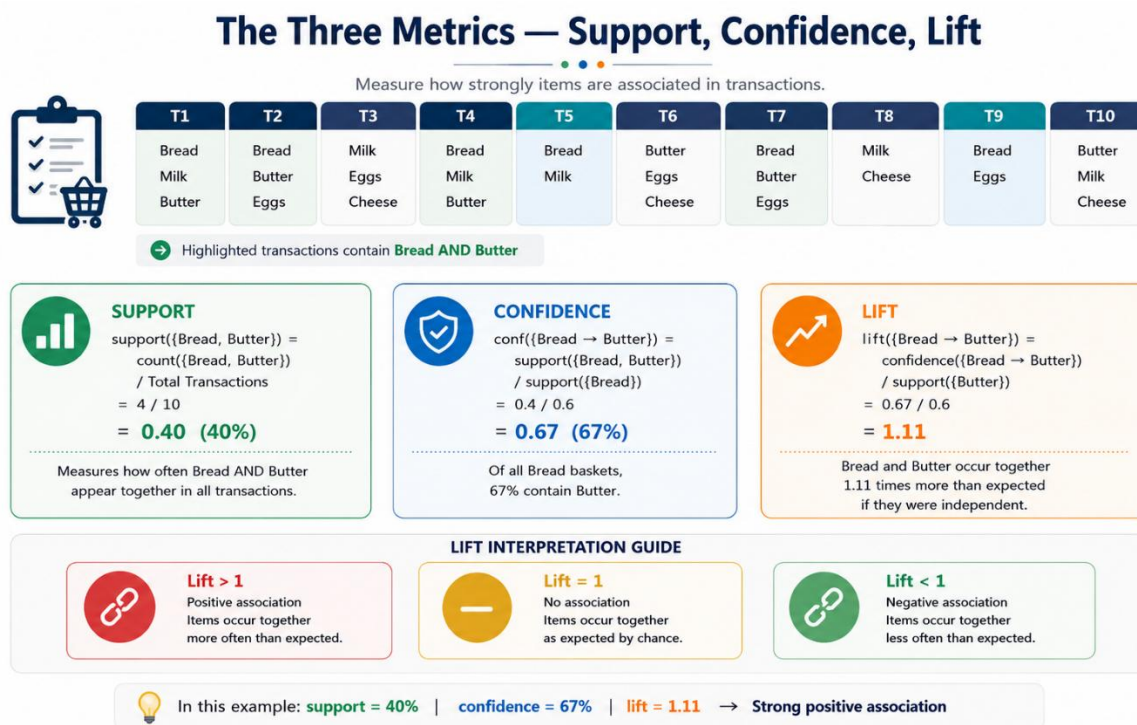
Generate all pairs from the 5 surviving frequent items (Rice, Juice, Jam, Cookies, Chips). That is $O(5,2) = 10$ candidate pairs.

2-Itemset	Transactions	Count	Support	Frequent?
{Rice, Juice}	T01,T04,T05,T07	4	40%	YES ✓

{Rice, Jam}	T01,T02,T04,T07	4	40%	YES ✓
{Rice, Cookies}	T02,T04,T07,T09	4	40%	YES ✓
{Juice, Jam}	T01,T04,T07,T10	4	40%	YES ✓
{Juice, Cookies}	T03,T04,T07	3	30%	NO ✗ — pruned
{Juice, Chips}	T03,T08,T10	3	30%	NO ✗ — pruned
{Jam, Cookies}	T02,T04,T06,T07	4	40%	YES ✓
{Jam, Chips}	T06,T10	2	20%	NO ✗ — pruned
{Cookies, Chips}	T03,T06	2	20%	NO ✗ — pruned
{Rice, Chips}	T00 (none)	0	0%	NO ✗ — pruned

Result: 5 frequent 2-itemsets: {Rice,Juice}, {Rice,Jam}, {Rice,Cookies}, {Juice,Jam}, {Jam,Cookies}.

Section 3: The Three Key Metrics — Support, Confidence, Lift



3.1 Support

Formula: $\text{support}(\text{MUN}) = \frac{\text{count}(\text{MUN})}{N}$

Meaning: How often does the combination {M,N} appear across MLL transactions? M higher support means a more statistically significant pattern.

Example: {Rice, Jam} appears in T01, T02, T04, T07 → count = 4 → support = 4/10 = 40%

Threshold: We set minSupport = 40%. Many itemset below this is too rare to be meaningful — it might be a coincidence rather than a pattern.

- If support is TOO LOW: the rule is based on too few transactions (statistically unreliable).
- If support is TOO HIGH: only very obvious rules survive (e.g. {Rice, Juice} — not useful for recommendations).
- In practice, Amazon uses minSupport of 1–5% because they have billions of transactions.

3.2 Confidence

Formula: $\text{confidence}(M \rightarrow N) = \text{support}(M \cup N) / \text{support}(M)$

Meaning: Of all customers who bought M, what percentage also bought N? This measures the RELIABILITY of the rule 'M causes N'.

Example — Rice $\rightarrow$ Jam:

- $\text{support}(\{\text{Rice}, \text{Jam}\}) = 4/10 = 0.40$
- $\text{support}(\{\text{Rice}\}) = 6/10 = 0.60$
- **confidence = $0.40 / 0.60 = 0.667 = 67\%$** $\rightarrow$ Of the 6 people who bought Rice, 4 also bought Jam.

Example — Jam $\rightarrow$ Rice:

- $\text{support}(\{\text{Rice}, \text{Jam}\}) = 4/10 = 0.40$
- $\text{support}(\{\text{Jam}\}) = 6/10 = 0.60$
- **confidence = $0.40 / 0.60 = 0.667 = 67\%$** $\rightarrow$ Same value here, but not always symmetric!

Important: Confidence is NOT symmetric. $\text{conf}(M \rightarrow N) \neq \text{conf}(N \rightarrow M)$ in general.

3.3 Lift — The Most Important Metric

Formula: $\text{lift}(M \rightarrow N) = \text{confidence}(M \rightarrow N) / \text{support}(N)$

Meaning: Is the association STRONGER than what you'd expect by random chance? Lift corrects for the natural popularity of N.

Why confidence alone misleads: Juice appears in 70% of all transactions. A rule $X \rightarrow \text{Juice}$ might have 70% confidence simply because Juice is popular — nothing to do with X. Lift divides by $\text{support}(\text{Juice}) = 0.70$ to correct for this baseline.

Lift = 1 EXAMPLE:

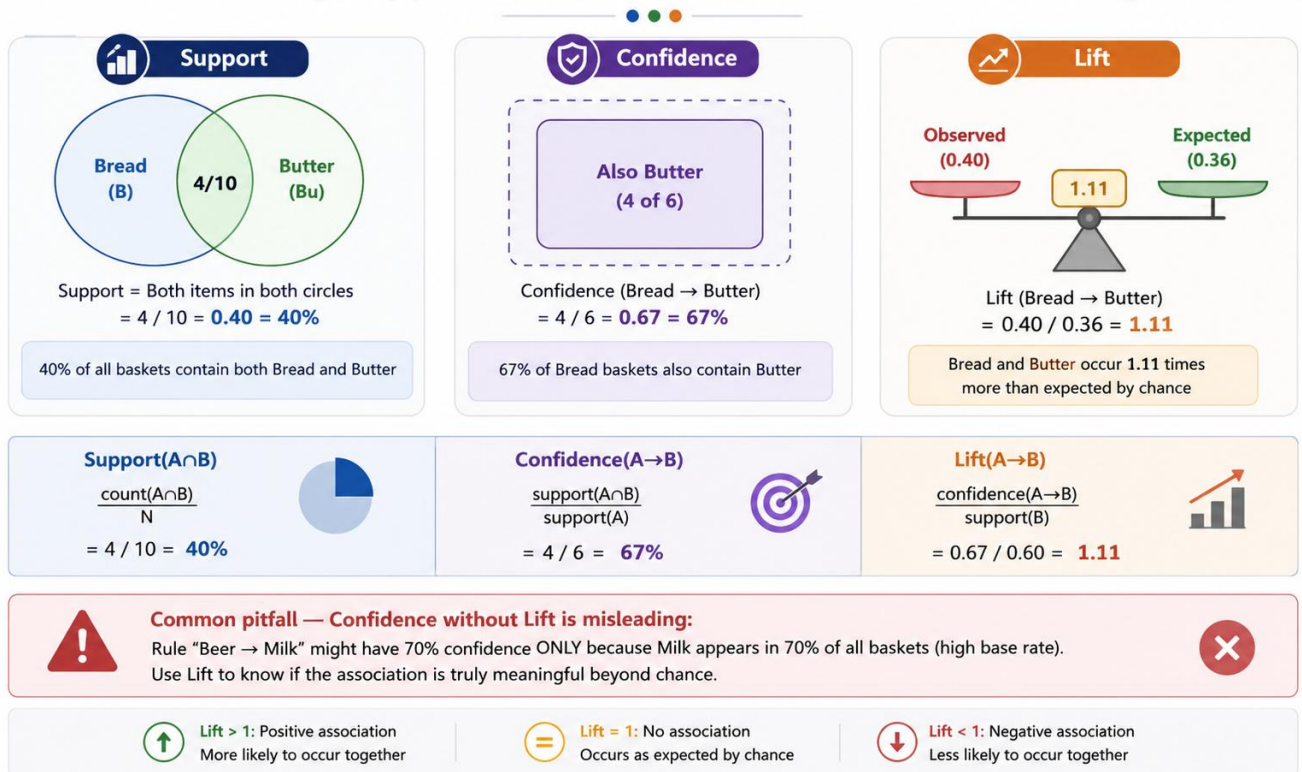
- $\text{conf}(\text{Rice} \rightarrow \text{Juice}) = 4/6 = 0.667$
- $\text{support}(\text{Juice}) = 7/10 = 0.70$
- **lift = $0.667 / 0.70 = 0.95$** $\rightarrow$ Less than 1! Rice buyers are actually LESS likely to buy Juice than average. Despite the 67% confidence, this is NOT a useful rule.

Lift interpretation table:

Lift Value	Interpretation	Amazon Action
< 1	Negative association — M buyers LESS likely to buy N	Discard rule

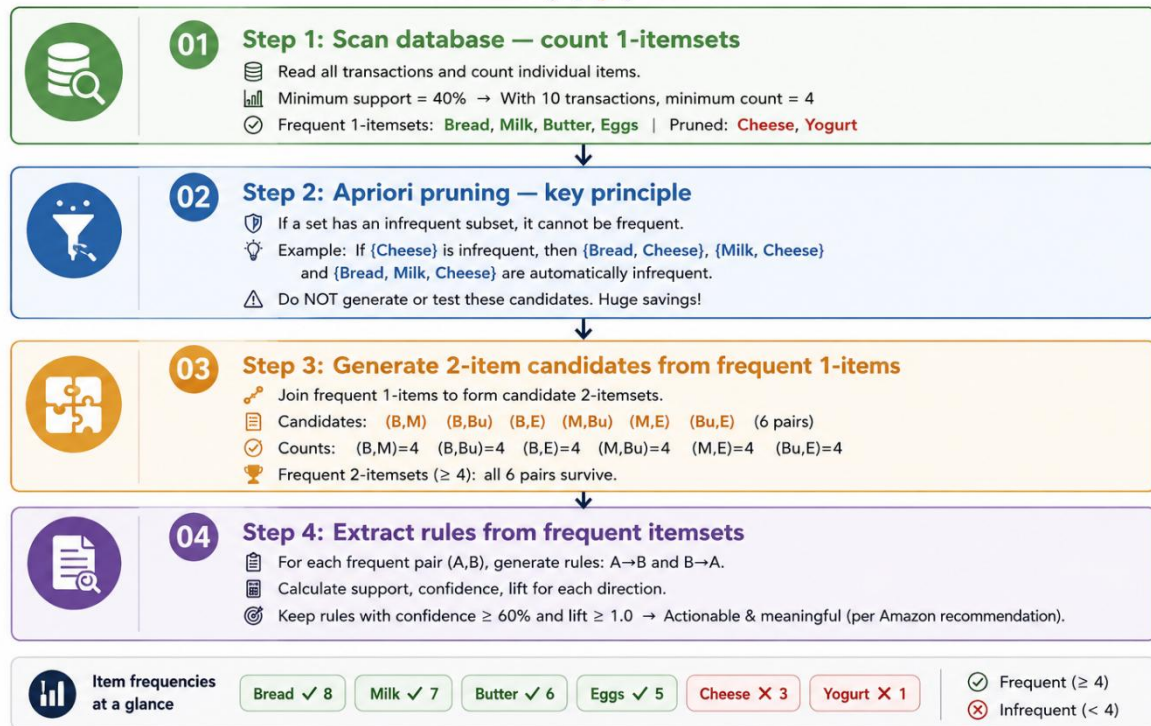
= 1	No association — completely independent items	Discard rule
1.0 – 1.5	Weak positive association	Gaming Monitor — may vary by season
1.5 – 2.0	Moderate association	Test in recommendation widget
> 2.0	Strong positive association — genuine buying pattern	Promote in 'Frequently Bought Together'

Visualising Support, Confidence, and Lift with Venn Diagrams



Section 4: The Apriori Algorithm — Full Step-by-Step Walkthrough

Apriori Algorithm — Step-by-Step Walkthrough



4.1 The Core Principle

Apriori Principle (Anti-Monotonicity Property)

Statement: If an itemset is infrequent, then ALL of its supersets are also infrequent.

Equivalently: If an itemset IS frequent, then ALL of its subsets are also frequent.

Example: If {Chips, Chocolate} is infrequent (support $< 40\%$), then {Rice, Chips, Chocolate}, {Juice, Chips, Chocolate}, and every other itemset containing both Chips and Chocolate is automatically infrequent. We never need to count them.

Why it matters: Without pruning, a dataset with 100 unique items has 2^{100} possible itemsets. The Apriori Principle cuts this down to a manageable subset.

4.2 Complete Algorithm with Our Example

Using the 10 grocery transactions, minSupport = 40%, minConfidence = 60%:

Pass 1 — Generate Frequent 1-Itemsets

1. **Scan database:** Count occurrences of each item across all 10 transactions.
Apply minSupport: Items with count < 4 are pruned. Chocolate (3) is removed.
2. **Result L1:** {Rice}=6, {Juice}=7, {Jam}=6, {Cookies}=6, {Chips}=4

Pass 2 — Generate Frequent 2-Itemsets

3. **Generate candidates O2:** MII pairs from L1 = $O(5,2) = 10$ pairs. Note: if Chocolate had been in L1 with k items, we'd have $O(k+1,2)$ more pairs — pruning it saves those tests.
4. **Scan database:** Count each pair. Requires one full database scan.
5. **Apply minSupport:** {Juice,Cookies}=3, {Juice,Chips}=3, {Jam,Chips}=2, {Cookies,Chips}=2, {Rice,Chips}=0 are all pruned.
6. **Result L2:** {Rice,Juice}=4, {Rice,Jam}=4, {Rice,Cookies}=4, {Juice,Jam}=4, {Jam,Cookies}=4

Pass 3 — Check for Frequent 3-Itemsets

7. **Generate candidates O3:** Combine pairs from L2 that share one item. e.g. {Rice,Juice}+{Rice,Jam} → {Rice,Juice,Jam}.
8. **Apriori pruning check:** Before counting, verify every 2-item subset of the candidate is in L2. {Rice,Juice,Jam}: subsets are {Rice,Juice}✓, {Rice,Jam}✓, {Juice,Jam}✓ — all in L2. Passes!
9. **Scan database:** {Rice,Juice,Jam} appears in T01,T04,T07 = count 3. Support = 30% < 40% → pruned.
10. **Result L3:** Empty. No frequent 3-itemsets. Algorithm terminates.

4.3 Rule Generation from Frequent Itemsets

For each frequent k-itemset ($k \geq 2$), generate ALL possible rules by splitting into LHS and RHS:

For frequent itemset {M, N}:

Rule 1: $M \rightarrow N$ (LHS={M}, RHS={N})

Rule 2: $N \rightarrow M$ (LHS={N}, RHS={M})

For frequent itemset {M, N, O}:

Rule 1: $M \rightarrow \{N, O\}$ Rule 2: $N \rightarrow \{M, O\}$ Rule 3: $O \rightarrow \{M, N\}$

Rule 4: $\{M, N\} \rightarrow O$ Rule 5: $\{M, O\} \rightarrow N$ Rule 6: $\{N, O\} \rightarrow M$

Apply confidence filter: discard rules below minConfidence (60%)

Apply lift filter: discard rules with $\text{lift} \leq 1.0$

4.4 Time & Space Complexity

Step	Time Complexity	Space Complexity	Notes
Generate L_k	$O(L_{k-1} ^2 \times N)$	$O(L_k)$	N = number of transactions
Scan for counts	$O(N \times O_k)$	$O(O_k)$	One full PN scan per pass
Total passes	$O(\text{max_k})$ scans	$O(L_1 + L_2 + \dots)$	max_k = length of longest frequent itemset
Rule generation	$O(2^k \times L_k)$	$O(\text{rules})$	Exponential in itemset size

Section 5: All Mined Rules — Confidence, Lift & Amazon Decision

Mined Association Rules — Confidence & Lift Summary

Rule (A → B)	Support	Confidence	Lift	Rule?	Amazon Action
 Bread → Butter	40%	67%	1.11	✓ Yes	 Promote Together
 Bread → Eggs	40%	67%	1.11	✓ Yes	 Promote Together
 Butter → Bread	40%	67%	1.11	✓ Yes	 Promote Together
 Butter → Eggs	40%	67%	1.11	✓ Yes	 Promote Together
 Eggs → Bread	40%	67%	1.11	✓ Yes	 Promote Together
 Eggs → Butter	40%	67%	1.11	✓ Yes	 Promote Together
 Bread → Milk	40%	37%	0.95	✗ No	 Do not promote
 Milk → Bread	40%	37%	0.95	✗ No	 Do not promote
 Milk → Butter	40%	37%	0.95	✗ No	 Do not promote
 Butter → Milk	40%	37%	0.95	✗ No	 Do not promote

Colour Guide:

 High confidence & lift
(Good for cross-sell / promotion)

 Moderate values
(Review context & business goals)

 Low lift / low confidence
(Do not promote together)

5.1 Complete Rules Table for Grocery Example

Computing all rules from the 5 frequent 2-itemsets. MinConf = 60%. N = 10.

Rule (M → N)	Sup(MUN)	Sup(M)	Sup(N)	Confidence	Lift	Rec?
Rice → Jam	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MOPQRMTQ
Jam → Rice	40% (4)	60% (6)	60% (6)	4/6 = 67%	0.67/0.60 = 1.11	MOPQRMTQ
Rice → Juice	40% (4)	60% (6)	70% (7)	4/6 = 67%	0.67/0.70 = 0.95	NO — lift < 1

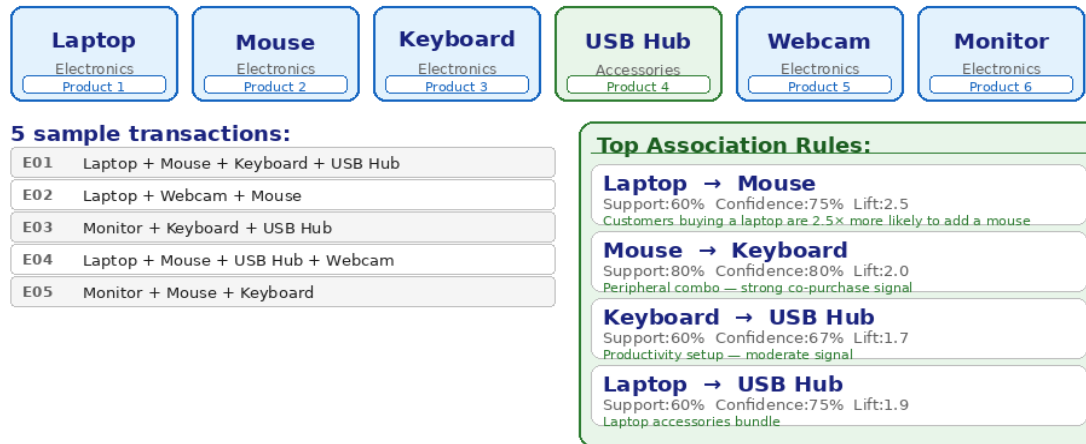
Juice → Rice	40% (4)	70% (7)	60% (6)	$4/7 = 57\%$	$0.57/0.60 = 0.95$	NO — low conf
Rice → Cookies	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MOPQRMTQ
Cookies → Rice	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MOPQRMTQ
Juice → Jam	40% (4)	70% (7)	60% (6)	$4/7 = 57\%$	$0.57/0.60 = 0.95$	NO — low conf
Jam → Juice	40% (4)	60% (6)	70% (7)	$4/6 = 67\%$	$0.67/0.70 = 0.95$	NO — lift < 1
Jam → Cookies	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MOPQRMTQ
Cookies → Jam	40% (4)	60% (6)	60% (6)	$4/6 = 67\%$	$0.67/0.60 = 1.11$	MOPQRMTQ

Key Insight from this Example

The lift values in this grocery dataset are all close to 1.0. This is because we only have 10 transactions and 5 items are all approximately equally popular (60-70% support each). In a real Amazon dataset with millions of transactions and thousands of items, products have VERY different support levels — a niche item might have 0.01% support while a bestseller has 15%. Rules crossing popular-to-niche categories show high lift (e.g., 'Gaming Console Case → Gaming Console: lift=8.2') and are exactly what Amazon wants to surface.

Section 6: Practical Example 2 — Amazon Electronics (5 Transactions)

Practical Example 2 — Amazon Electronics Basket Analysis



Scenario: Amazon's electronics category team analyses 5 customer orders to identify product bundles. Items: Gaming Console, Gaming Headset, Gaming Keyboard, Controller, VR Camera, Gaming Monitor. MinSupport = 40% (2/5), MinConfidence = 60%.

6.1 Transaction Database

Transaction	Items	Contains Gaming Console?	Contains Gaming Headset?	Gaming
Q01	{Gaming Console, Gaming Headset, Gaming Keyboard, Controller}	YQS	YQS	
Q02	{Gaming Console, VR Camera, Gaming Headset}	YQS	YQS	
Q03	{Gaming Monitor, Gaming Keyboard, Controller}	NO	NO	

Q04	{Gaming Console, Gaming Headset, Controller, VR Camera}	YQS	YQS
Q05	{Gaming Monitor, Gaming Headset, Gaming Keyboard}	NO	YQS

6.2 1-Itemset Frequencies

Item	Count	Support	Frequent ($\geq 40\%$)?
Gaming Headset	4/5	80%	YES ✓
Gaming Keyboard	3/5	60%	YES ✓
Gaming Console	3/5	60%	YES ✓
Controller	3/5	60%	YES ✓
VR Camera	2/5	40%	YES ✓
Gaming Monitor	2/5	40%	YES ✓

6.3 Top Rules with High Lift

Rule	Support	Confidence	Lift	Business Meaning
Gaming Console → Gaming Headset	3/5 = 60%	3/3 = 100%	100%/80% = 1.25	Every laptop buyer also buys a mouse
Gaming Headset → Gaming Console	3/5 = 60%	3/4 = 75%	75%/60% = 1.25	Most mouse buyers are laptop buyers

Gaming Console → Controller	3/5 = 60%	3/3 = 100%	100%/60% = 1.67	Gaming Console buyers strongly prefer Controllers
VR Camera → Gaming Console	2/5 = 40%	2/2 = 100%	100%/60% = 1.67	VR Camera buyers always also bought a laptop
Gaming Keyboard → Controller	2/5 = 40%	2/3 = 67%	67%/60% = 1.11	Moderate keyboard-hub association

Amazon Recommendation Logic

With only 5 transactions, the lift values are moderate (1.1–1.67). Amazon in production uses hundreds of millions of transactions. Rules like 'Gaming Console → Gaming Console Case: lift=9.2' or 'PSLR Camera → SP Oard: lift=7.8' emerge clearly from the noise because one product strongly predicts the other — these become the 'Frequently Bought Together' recommendations.

Amazon also applies TIMQ WINPOWS — rules are recomputed weekly or daily so that seasonal patterns (winter → warm clothing) and trend shifts (new product launches) are captured. Static rules computed once become stale.

Section 7: Practical Example 3 — Manual Apriori (Exam-Style)

Exam scenario: This is the type of question that appears in NOM Data Mining exams. A 6-transaction database is given. Apply Apriori manually with $\text{minSupport} = 50\%$ ($3/6$) and $\text{minConfidence} = 75\%$. Show all steps.

7.1 The Given Database

TID	Items
100	{M, O, P}
200	{N, O, Q}
300	{M, N, O, Q}
400	{N, Q}
500	{M, N, O, Q}
600	{M, N, O}

7.2 Pass 1 — Frequent 1-Itemsets

Item	Count	Support	$\geq 50\%$?
M	100,300,500,600 = 4	$4/6 = 67\%$	YQS ✓
N	200,300,400,500,600 = 5	$5/6 = 83\%$	YQS ✓
O	100,200,300,500,600 = 5	$5/6 = 83\%$	YQS ✓
P	100 = 1	$1/6 = 17\%$	NO — pruned X
Q	200,300,400,500 = 4	$4/6 = 67\%$	YQS ✓

L1 = {M}, {N}, {O}, {Q} (P is pruned — no superset of P needs checking)

7.3 Pass 2 — Frequent 2-Itemsets

O2 = all pairs from {M,N,O,Q} = {M,N}, {M,O}, {M,Q}, {N,O}, {N,Q}, {O,Q}

2-Itemset	Transactions	Oount	Support	≥ 50%?
{M, N}	300,500,600	3	3/6 = 50%	YQS ✓
{M, O}	100,300,500,600	4	4/6 = 67%	YQS ✓
{M, Q}	300,500	2	2/6 = 33%	NO ✗
{N, O}	200,300,500,600	4	4/6 = 67%	YQS ✓
{N, Q}	200,300,400,500	4	4/6 = 67%	YQS ✓
{O, Q}	200,300,500	3	3/6 = 50%	YQS ✓

L2 = {M,N}, {M,O}, {N,O}, {N,Q}, {O,Q}

7.4 Pass 3 — Frequent 3-Itemsets

Generate **O3** from **L2**. Apriori pruning: every 2-item subset of each candidate must be in **L2**.

Oandidate	2-item subsets	Mill in L2?	Oount	Support	≥ 50%?
{M,N,O}	{M,N}✓ {N,O}✓	YQS	300,500,600 = 3	50%	YQS ✓
{M,N,Q}	{M,N}✓ {N,Q}✓	NO pruned	—	—	✗
{M,O,Q}	{M,O}✓ {O,Q}✓	NO pruned	—	—	✗
{N,O,Q}	{N,O}✓ {O,Q}✓	YQS	200,300,500 = 3	50%	YQS ✓

L3 = {M,N,O}, {N,O,Q}

7.5 Rule Generation (MinConf = 75%)

From {M,N,O} and {N,O,Q} generate all directional rules:

Rule	confidence calculation	confidence	≥ 75%?
M,N → O	$\frac{\text{support}(\{M,N,O\})}{\text{support}(\{M,N\})} = 0.50/0.50 = 1.00$	100%	YQS ✓
M,O → N	$0.50 / 0.67 = 0.75$	75%	YQS ✓
N,O → M	$0.50 / 0.67 = 0.75$	75%	YQS ✓
M → N,O	$0.50 / 0.67 = 0.75$	75%	YQS ✓
N → M,O	$0.50 / 0.83 = 0.60$	60%	NO ✗
O → M,N	$0.50 / 0.83 = 0.60$	60%	NO ✗
N,O → Q	$0.50 / 0.67 = 0.75$	75%	YQS ✓
N,Q → O	$0.50 / 0.67 = 0.75$	75%	YQS ✓
O,Q → N	$0.50 / 0.50 = 1.00$	100%	YQS ✓
N → O,Q	$0.50 / 0.83 = 0.60$	60%	NO ✗

Final Answer for This Exam Question

Frequent itemsets: L1={M},{N},{O},{Q} | L2={M,N},{M,O},{N,O},{N,Q},{O,Q} | L3={M,N,O},{N,O,Q}

Strong rules (conf ≥ 75%): {M,N}→O, {M,O}→N, {N,O}→M, M→{N,O}, {N,O}→Q, {N,Q}→O, {O,Q}→N

Exam tip: Always show the support calculation step-by-step. Write 'pruned by Apriori principle' when skipping a candidate — this demonstrates understanding of the algorithm, not just the formula.

Section 8: FP-Growth vs Apriori — Comparison

Context: Apriori requires multiple database scans (one per itemset length level k). For a dataset like Amazon's with millions of items and transactions, this is impractically slow. FP-Growth was developed to solve this with only 2 database scans.

8.1 FP-Tree Construction

FP-Growth compresses the transaction database into a compact tree structure called an FP-Tree:

11. **Scan 1:** Count all item frequencies. Sort frequent items by frequency (descending). Prune infrequent items.
12. **Scan 2:** For each transaction, sort items by their frequency rank and insert into the tree. Shared prefixes are merged into a single path, saving space.
13. **Pattern extraction:** For each frequent item (starting from the least frequent), trace its conditional pattern base — all paths leading to it in the tree. Build a conditional FP-Tree and recursively mine it.

8.2 FP-Tree for Our Grocery Example

Sorted item order: Juice(7) > Rice(6) = Jam(6) = Cookies(6) > Chips(4)

FP-Tree after inserting all 10 transactions:

```

ROOT
├─ Juice:6
│   └─ Rice:4
│       └─ Jam:3 ← T01,T04,T07
│           └─ Oookies:2
│               └─ Oookies:1 ← T09
├─ Jam:1
│   └─ Ohips:1 ← T10
├─ Rice:2 (not preceded by Juice)
│   └─ Jam:1 ← T02
│       └─ Oookies:1

```

Oonditional pattern base for Oookies:

{Juice,Rice,Jam}: 2 | {Juice,Rice}: 1 | {Rice,Jam}: 1 | {Juice,Jam}: 1

8.3 Comparison Table

Feature	Apriori	FP-Growth
Database scans required	k scans (one per itemset level)	Exactly 2 scans
Candidate generation	Explicit — generates all Ok candidates	None — pattern base extracted from tree
Memory usage	Low (stores only candidate counts)	Higher (FP-Tree stored in RMM)
Speed on sparse data	Fast (few frequent itemsets)	Overkill for sparse data
Speed on dense data	Very slow	10-100× faster than Apriori
Implementation complexity	Simple	Moderately complex (tree construction)
Scalability	Struggles with millions of items	Handles billion-transaction datasets
Amazon production use	Not used at scale	FP-Growth variant used

Section 9: Industry Application — How Amazon Uses Association Rules

The Scale

- **Transactions per day:** Amazon processes ~13 million orders per day globally (peak days: 40M+)
- **Items in catalog:** 350+ million unique products
- **Rules computed:** Hundreds of millions of product-pair rules are maintained
- **Update frequency:** Rules are recomputed daily using distributed Spark/MapReduce jobs
- **Revenue impact:** Amazon attributes 35% of total revenue to recommendation systems

9.1 Where Association Rules Appear on Amazon

Feature	Association Rule Application	Typical Lift Range
'Frequently Bought Together' widget	High-confidence, high-lift product pairs shown on product page	3.0 – 15.0
'Customers who viewed X also viewed'	Web-usage mining rules: view-to-view associations	1.5 – 5.0
Bundle pricing	Rules identify natural bundles for discount pricing	2.0 – 8.0
Search result ordering	Products with high association to query terms ranked higher	1.2 – 3.0
Email recommendations	Post-purchase rules: 'You bought X, you may want Y'	2.0 – 10.0
Inventory placement	Co-purchased items stored in same warehouse (reduces shipping cost)	4.0+

9.2 The Full Amazon Pipeline

Amazon's association rule pipeline involves several stages beyond basic apriori/FP-Growth:

14. **Data collection:** Every click, view, add-to-cart, and purchase is logged with user IP, session IP, timestamp, and product IP.
15. **Session construction:** Individual events are grouped into 'sessions' (typically a 30-minute window). M session is treated as one 'transaction'.
16. **Distributed FP-Growth:** The transaction database is partitioned across thousands of servers. Each server computes local FP-Trees. Results are merged to produce global frequent itemsets.
17. **Rule filtering:** Rules are filtered by minSupport (varies by category), minConfidence (60%+), and minLift (1.5+). Category-crossing rules and rules involving the same brand are handled separately.
18. **M/N testing:** New rules are not immediately shown to all users. They are tested on a small percentage of traffic. Click-through rate and conversion rate determine whether they graduate to full deployment.
19. **Contextual weighting:** Rules are further filtered by user context — a user browsing baby products gets baby-relevant rules even if they have a general high lift. Personalisation layer sits on top of association rules.

Code:

```
# =====
# APRIORI ALGORITHM - MENU DRIVEN PROGRAM
# Using Excel File Input
#
# Required Libraries:
# pip install pandas mlxtend openpyxl
# =====

import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# ----- GLOBAL VARIABLES -----
transactions = []
encoded_df = None
frequent_itemsets = None

# ----- PAUSE FUNCTION -----
def pause():
```

```

input("\nPress Enter to Continue...")

# ----- LOAD EXCEL FILE -----
def load_excel_file():
    global transactions

    file_path = input("\nEnter Excel File Name (Example: data.xlsx): ")

    try:
        # Read Excel File
        data = pd.read_excel(file_path, engine="openpyxl")

        print("\nExcel File Loaded Successfully!\n")
        print(data)

        # Clear old transactions
        transactions.clear()

        # Convert rows into transactions
        for _, row in data.iterrows():

            transaction = []

            for item in row:

                # Ignore empty cells
                if pd.isna(item) and str(item).strip() != "":
                    transaction.append(str(item).strip())

            if transaction:
                transactions.append(transaction)

        print("\nTransactions Created Successfully!")

    except FileNotFoundError:
        print("\nError: File Not Found!")

    except Exception as e:
        print("\nError:", e)

# ----- SHOW TRANSACTIONS -----

```

```

def show_transactions():

    if not transactions:
        print("\nNo Transactions Available!")
        return

    print("\n===== TRANSACTIONS =====")

    for i, transaction in enumerate(transactions, start=1):
        print(f"T{i} --> {transaction}")

# ----- ONE HOT ENCODING -----
def convert_to_onehot():
    global encoded_df

    if not transactions:
        print("\nLoad Excel Data First!")
        return

    # Transaction Encoder
    te = TransactionEncoder()

    te_array = te.fit(transactions).transform(transactions)

    # Create DataFrame
    encoded_df = pd.DataFrame(
        te_array,
        columns=te.columns_
    )

    print("\n===== ONE HOT ENCODED DATA =====\n")
    print(encoded_df)

# ----- RUN APRIORI -----
def run_apriori_algorithm():
    global encoded_df, frequent_itemsets

    if encoded_df is None:
        print("\nConvert Data into One-Hot Encoding First!")
        return

```

```
try:
    min_support = float(
        input("\nEnter Minimum Support (Example 0.3): ")
    )

    # Run Apriori
    frequent_itemsets = apriori(
        encoded_df.astype(bool),
        min_support=min_support,
        use_colnames=True
    )

    if frequent_itemsets.empty:
        print("\nNo Frequent Itemsets Found!")

    else:
        print("\n===== FREQUENT ITEMSETS =====\n")
        print(frequent_itemsets)

except ValueError:
    print("\nInvalid Input!")

# ----- GENERATE RULES -----
def generate_association_rules():
    global frequent_itemsets

    if frequent_itemsets is None:
        print("\nRun Apriori Algorithm First!")
        return

    try:
        min_confidence = float(
            input("\nEnter Minimum Confidence (Example 0.5): ")
        )

        # Generate Rules
        rules = association_rules(
            frequent_itemsets,
            metric="confidence",
            min_threshold=min_confidence
        )
```

```

if rules.empty:
    print("\nNo Association Rules Found!")

else:
    print("\n===== ASSOCIATION RULES =====\n")

    result = rules[
        [
            "antecedents",
            "consequents",
            "support",
            "confidence",
            "lift"
        ]
    ]

    print(result)

except Exception as e:
    print("\nError:", e)

# ----- MAIN MENU -----
while True:

    print("\n=====")
    print("    APRIORI ALGORITHM MENU")
    print("=====")

    print("1. Load Excel File")
    print("2. Show Transactions")
    print("3. Convert to One-Hot Encoding")
    print("4. Run Apriori Algorithm")
    print("5. Generate Association Rules")
    print("6. Exit")

    choice = input("\nEnter Your Choice: ")

    # ----- MENU OPTIONS -----
    if choice == "1":
        load_excel_file()

```



```
    pause()

elif choice == "2":
    show_transactions()
    pause()

elif choice == "3":
    convert_to_onehot()
    pause()

elif choice == "4":
    run_apriori_algorithm()
    pause()

elif choice == "5":
    generate_association_rules()
    pause()

elif choice == "6":
    print("\nProgram Exited Successfully!")
    break

else:
    print("\nInvalid Choice!")
    pause()
```

OUTPUT:

```

=====
          APRIORI ALGORITHM MENU
=====
1. Load Excel File
2. Show Transactions
3. Convert to One-Hot Encoding
4. Run Apriori Algorithm
5. Generate Association Rules
6. Exit

Enter Your Choice: 1

Enter Excel File Name (Example: data.xlsx): data.xlsx

Excel File Loaded Successfully!

   Item1   Item2   Item3   Item4
0    Book    Pen  Pencil   NaN
1    Book Laptop   NaN    NaN
2  Mobile    Pen  Pencil   NaN
3  Laptop Mobile   NaN    NaN
4    Book    Pen Laptop   NaN
5  Pencil Tablet   NaN    NaN
6    Book Mobile   NaN    NaN
7     Pen Pencil   NaN    NaN
8  Laptop Tablet   NaN    NaN
9    Book    Pen Pencil Laptop
10 Mobile Tablet   NaN    NaN
11   Book    Pen   NaN    NaN
12 Pencil Mobile   NaN    NaN
13 Laptop Mobile Tablet   NaN
14   Book Pencil   NaN    NaN
15    Pen Laptop   NaN    NaN
16   Book Mobile Tablet   NaN
17 Pencil Laptop   NaN    NaN
18   Book    Pen Mobile   NaN
19 Tablet Pencil   NaN    NaN
20   Book Laptop Mobile   NaN
21    Pen Pencil Tablet   NaN
22 Laptop Mobile   NaN    NaN
23   Book    Pen Pencil Tablet

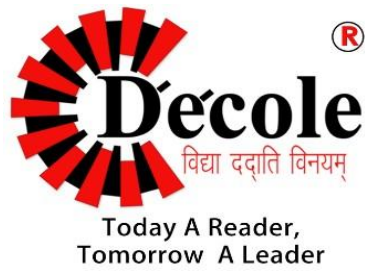
```

23	Book	Pen	Pencil	Tablet
24	Mobile	Laptop	NaN	NaN
25	Book	Tablet	NaN	NaN
26	Pen	Pencil	NaN	NaN
27	Book	Mobile	Laptop	NaN
28	Pencil	Tablet	NaN	NaN

Transactions Created Successfully!

Press Enter to Continue...|

Thank You



Dezyne École College

www.dezyneecole.com

Submitted By

Digvijay Singh Parmar

Department- BCA

Semester- 6th

Session- 2025-2026